

A PROJECT REPORT ON

AI RESUME SCREENING AND JOB RECOMMENDATION PLATFORM

Domain: NLP / Recruitment

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 Software Specification	
V	SYSTEM ANALYSIS	10
	5.1 Feasibility Study	

	5.2 About the Software	
VI	SYSTEM DESIGN	14
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	23
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	30
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	31

CHAPTER I

INTRODUCTION

Every year lakhs of fresh graduates enter the job market in India. Many of them apply to hundreds of jobs without receiving any response and do not know why their resumes are rejected. At the same time, recruiters receive hundreds or thousands of applications for a single opening and spend hours reading resumes, often rejecting good candidates because of time pressure.

Most companies use Applicant Tracking Systems (ATS) that filter resumes by counting keywords. Keyword matching misses equivalent skills written differently, for example 'ML' and 'machine learning', or 'React.js' and 'ReactJS', and it cannot understand the context of a candidate's experience. Candidates, on the other hand, receive no feedback about missing skills.

Natural language processing techniques such as named entity recognition and sentence embeddings allow a system to understand resumes and job descriptions semantically. This project, HireMatch, lets candidates upload a resume and receive an ATS score, missing-skill suggestions and matching job listings, while recruiters receive a ranked list of candidates for a job description based on semantic similarity rather than keyword counting.

LIMITATIONS

1. Fresh graduates do not know why their resumes get rejected.
2. Recruiters spend hours filtering hundreds of resumes.
3. Keyword matching misses equivalent skills.
4. There is no link between skill gaps and learning resources.

1.1 History of Domain

Recruitment moved from newspaper advertisements and paper applications to online job portals in the late 1990s (Monster, Naukri.com in 1997). As the volume of

online applications grew, companies adopted Applicant Tracking Systems that store applications and filter them using Boolean keyword search.

Information retrieval methods such as TF-IDF and BM25 improved ranking of resumes. Since 2018, transformer models and sentence embeddings (Sentence-BERT, 2019) have allowed semantic matching of resumes and job descriptions, and skill taxonomies such as ESCO and O*NET help normalise skill names.

1.2 Overview of the Project

HireMatch parses an uploaded PDF or DOCX resume into sections (education, skills, experience, projects, certifications) using layout-aware text extraction and heading detection. A spaCy NER pipeline with an EntityRuler built from a skill ontology extracts skills, degrees, institutions and years of experience. Skills are normalised to canonical names.

The resume and the job description are embedded using Sentence-BERT. The final score combines semantic similarity, the percentage of required skills matched and experience/education rules. Candidates see an ATS score with missing skills and recommended free courses; recruiters see candidates ranked for each job with an explanation of matched and missing skills.

1.3 Motivation

Talented freshers from smaller towns often miss opportunities simply because their resumes are not written in the way ATS software expects.

- **Motivation:** To make screening fair and transparent by understanding meaning instead of counting keywords.
- **Outcome:** Improved employability of freshers through feedback, and faster, better shortlisting for recruiters.

1.4 Aim of the Project

The project is entitled as “**AI RESUME SCREENING AND JOB RECOMMENDATION PLATFORM**”. The aim of the project is to develop an NLP-based platform that parses resumes, extracts and normalises skills, computes semantic match scores with job descriptions, identifies skill gaps, recommends jobs and courses, and ranks candidates for recruiters.

1.5 Objectives

- To parse PDF and DOCX resumes into structured sections.
- To extract skills, education and experience using NER and a skill ontology.
- To compute semantic similarity using Sentence-BERT.
- To identify skill gaps and recommend courses and jobs.
- To rank candidates for recruiters with explanations.

1.6 Literature Review

Robertson and Zaragoza (2009): Described the probabilistic relevance framework and the BM25 ranking function, widely used as a strong baseline for document retrieval including resume search.

Mikolov et al. (2013): Proposed word2vec, efficient estimation of word representations in vector space, showing that semantically similar words have similar vectors.

Reimers and Gurevych (2019): Proposed Sentence-BERT, which uses siamese BERT networks to derive semantically meaningful sentence embeddings that can be compared using cosine similarity, reducing similarity search time dramatically.

Honnibal et al. (spaCy, 2020): spaCy provides industrial-strength NLP pipelines with tokenisation, tagging, parsing and named entity recognition with support for rule-based entity matching.

Raghavan et al. (2020): Studied algorithmic pre-employment assessments and highlighted the need for transparency and bias mitigation in automated hiring tools.

ESCO (European Commission): The European Skills, Competences, Qualifications and Occupations classification provides a multilingual skill taxonomy useful for skill normalisation.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

In the existing system candidates submit resumes on job portals and receive no feedback. Recruiters use ATS keyword filters or read resumes manually. Resume checker websites provide generic formatting tips without matching to a specific job description.

DISADVANTAGES

- Keyword filters reject good candidates who use different words.
- Manual screening is slow and inconsistent.
- Candidates get no feedback about skill gaps.
- No learning recommendations.
- Recruiters cannot see why a candidate matches.

2.2 Proposed System

The proposed HireMatch platform uses NLP to parse resumes, extract and normalise skills with an ontology, and compute semantic similarity between resumes and job descriptions using Sentence-BERT. It provides candidates with an explainable ATS score, skill gaps and course recommendations, and provides recruiters with a ranked, explained candidate list.

ADVANTAGES

- Semantic matching recognises equivalent skills.
- Explainable scores with matched and missing skills.
- Course recommendations to close skill gaps.
- Screening time reduced from hours to minutes.
- Same platform benefits both candidates and recruiters.

2.3 Project Schedule

The development of HireMatch follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the HireMatch system are listed below.

- **Resume upload:** Accept PDF and DOCX resumes up to 5 MB.
- **Resume parsing:** Split resume into sections and extract text.
- **Entity extraction:** Extract skills, degrees, colleges, experience.
- **Skill normalisation:** Map synonyms to canonical skills.
- **Scoring:** Compute ATS score against a job description.
- **Gap analysis:** List missing required skills.
- **Recommendations:** Recommend jobs and courses.
- **Recruiter ranking:** Rank all applicants for a job with explanation.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Performance:** Score a resume within 3 seconds; rank 500 resumes within 1 minute.
- **Fairness:** Name, gender, age and photo are excluded from scoring.

- **Privacy:** Resumes visible only to recruiters the candidate applied to.
- **Usability:** Clear score breakdown and suggestions.
- **Scalability:** Embeddings cached for fast re-ranking.
- **Security:** JWT authentication and role-based access.

3.3 User Requirements

- Candidates get a score and specific suggestions
- Candidates see matching jobs
- Recruiters post job descriptions
- Recruiters see ranked candidates with reasons
- Simple dashboard for both roles

3.4 System Requirements

- FastAPI server with spaCy and Sentence-Transformers
- PostgreSQL database
- React web front end
- File storage for resumes
- Skill ontology and course catalogue

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System : Intel i5 Processor 2.4 GHz or above

Hard Disk : 256 GB SSD

RAM : 8 GB

Monitor : 15" colour monitor

4.2 Software Specification

Operating : Windows 10 / Ubuntu 22.04

System

Coding : Python 3.10, JavaScript

Language

NLP Libraries : spaCy, Sentence-Transformers, pdfplumber, python-docx

Back End : FastAPI

Front End : React JS

Database : PostgreSQL

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed HireMatch system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

All NLP components are available as open-source libraries with pre-trained models. Sentence-BERT models run efficiently on CPU. The project is technically feasible.

ECONOMIC FEASIBILITY

The system can be hosted on a low-cost server and uses free libraries. It saves significant recruiter time, so it is economically feasible for colleges' placement cells and small companies.

OPERATIONAL FEASIBILITY

Candidates upload a file and read suggestions; recruiters paste a job description. The system fits naturally into existing placement workflows, so it is operationally feasible.

5.2 About the Software

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an

ecosystem of third-party packages available through the Python Package Index (PyPI), which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

SPACY

spaCy is an open-source library for advanced natural language processing in Python. It provides fast tokenisation, part-of-speech tagging, dependency parsing and named entity recognition (NER).

Custom entity types such as SKILL or DEGREE can be added using the EntityRuler with pattern lists or by training a statistical NER component.

SENTENCE-TRANSFORMERS (SBERT)

Sentence-Transformers is a Python framework for state-of-the-art sentence, paragraph and document embeddings. It modifies BERT using siamese and triplet network structures so that semantically similar sentences are placed close together in vector space.

Similarity between two texts is calculated using cosine similarity of their embeddings, which captures meaning rather than exact words; for example 'ML engineer' and 'machine learning developer' get a high score.

FASTAPI

FastAPI is a modern, high-performance web framework for building APIs with Python based on standard Python type hints. It is built on Starlette for the web parts and Pydantic for data validation.

FastAPI automatically generates interactive API documentation (Swagger UI and ReDoc), validates request bodies, and supports asynchronous request handling, which gives performance comparable to Node.js and Go. It is well suited for serving machine learning models with low latency.

- Automatic request validation with Pydantic
- Auto-generated OpenAPI documentation
- Asynchronous and very fast

REACT JS

React is an open-source JavaScript library for building user interfaces, maintained by Meta and a community of developers. It lets developers build reusable UI components and manages the view layer of web applications efficiently using a virtual DOM.

React applications are composed of components that hold their own state and re-render automatically when data changes. With libraries such as React Router and Axios, a single-page application can communicate with REST APIs and give the user a fast, app-like experience.

- Component based, reusable UI
- Virtual DOM for fast rendering
- Large ecosystem of UI libraries

POSTGRESQL

PostgreSQL is a powerful, open-source object-relational database system with over 35 years of active development. It has earned a strong reputation for reliability, feature robustness and performance.

PostgreSQL supports ACID transactions, foreign keys, views, triggers, stored procedures and JSON data types. Its extension system allows additional capabilities such as PostGIS for geographic data and pgvector for similarity search.

- Fully ACID compliant
- Advanced SQL and JSON support

- Extensible with PostGIS and other extensions

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of HireMatch is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

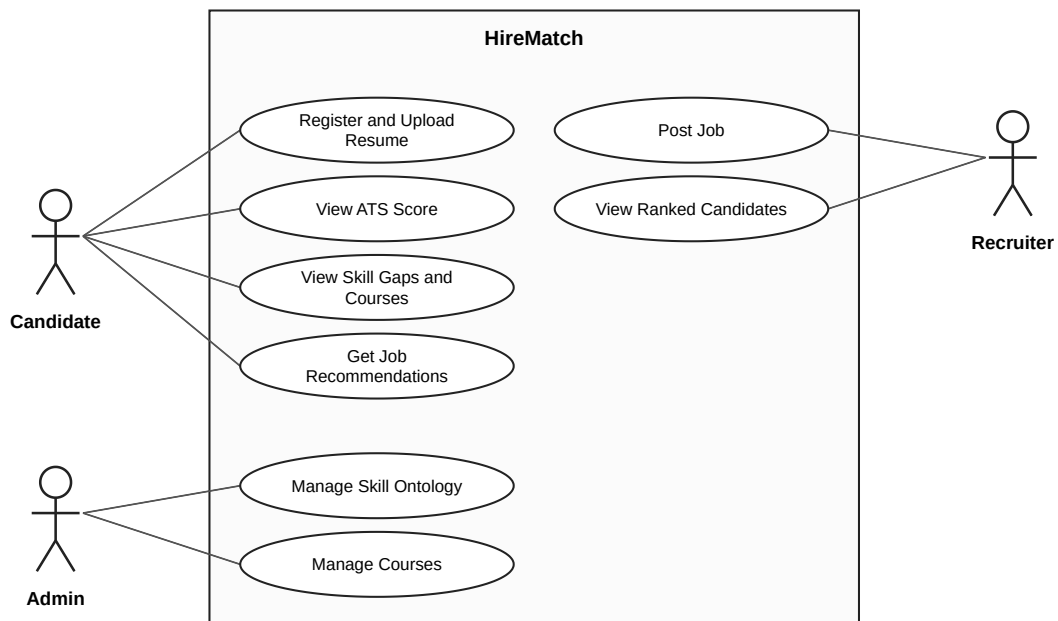


Fig 1: Use case diagram of HireMatch

6.2 System Architecture

HireMatch follows a three-tier architecture with an NLP processing pipeline in the application tier.

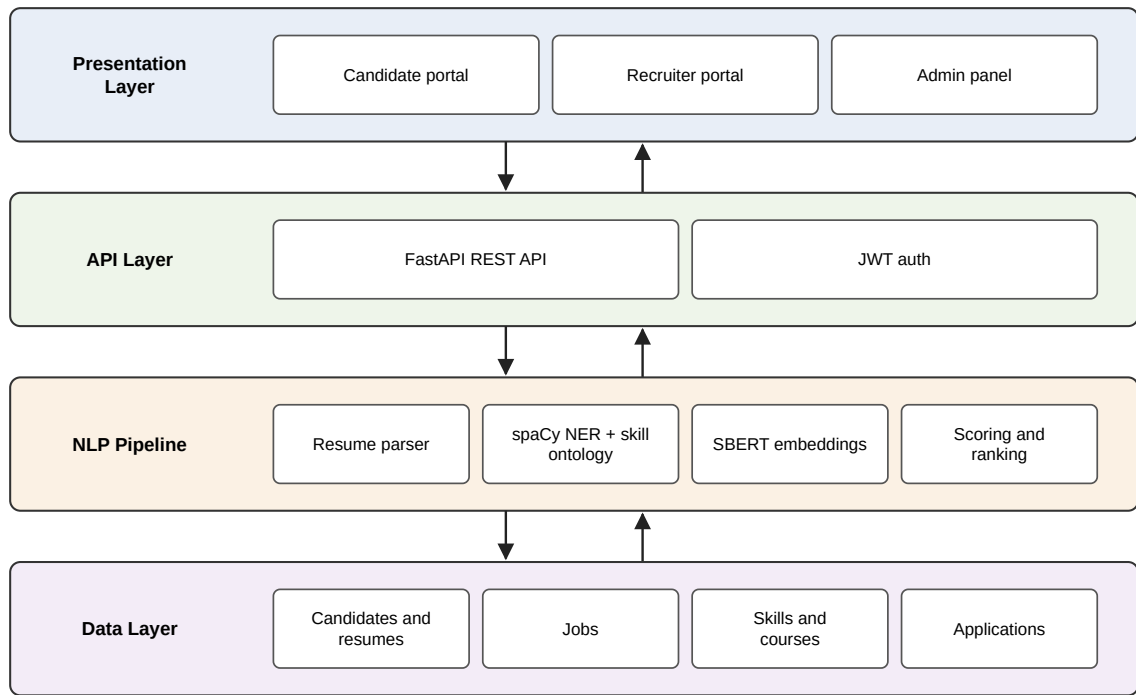


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

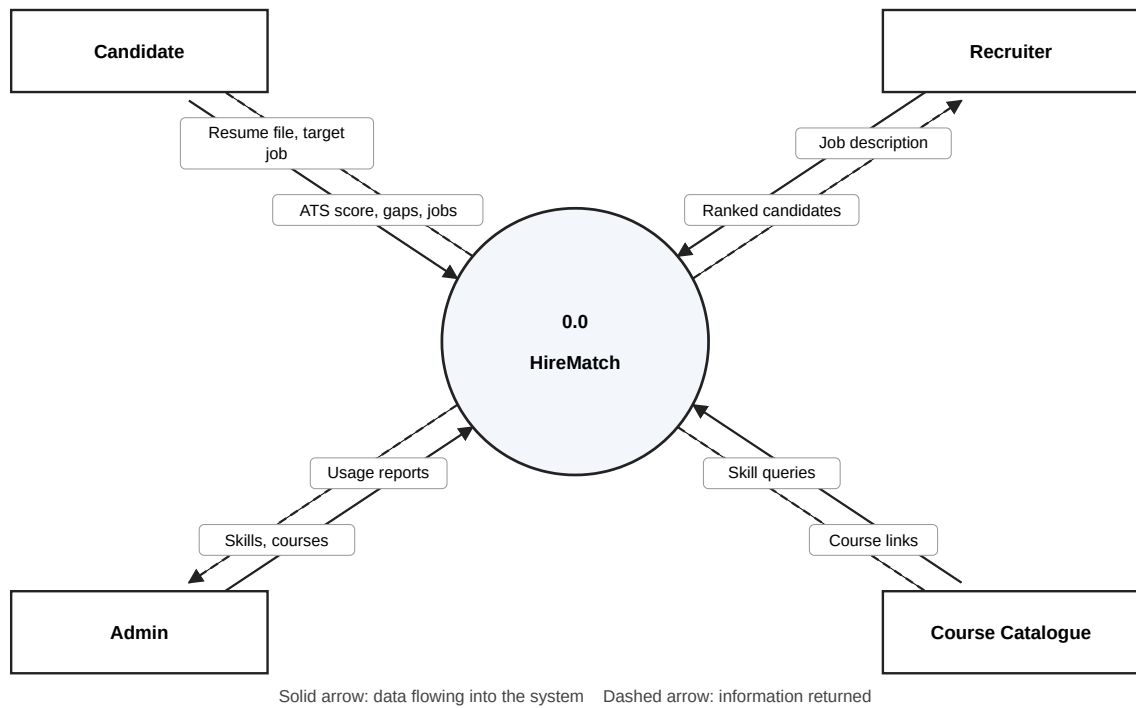


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

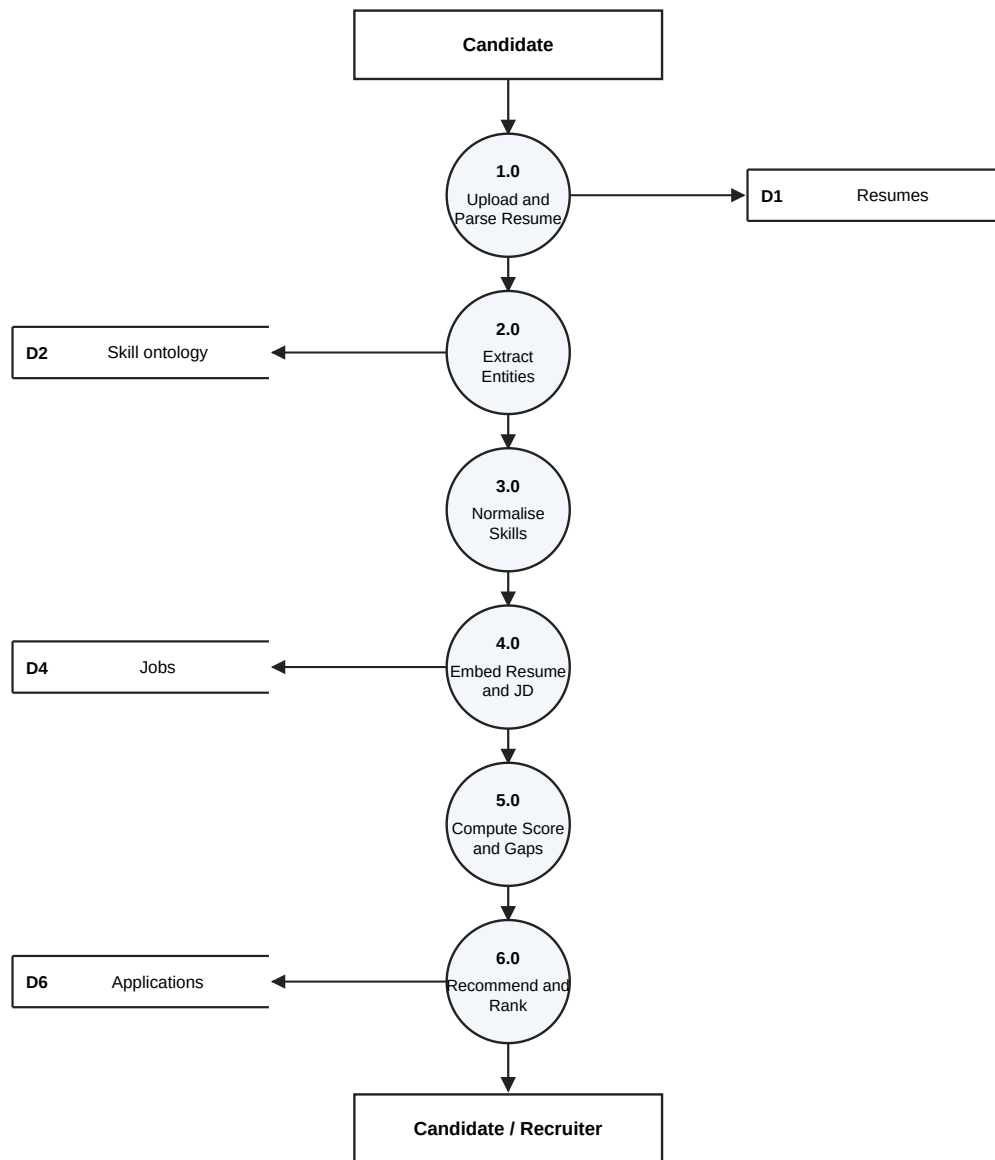


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

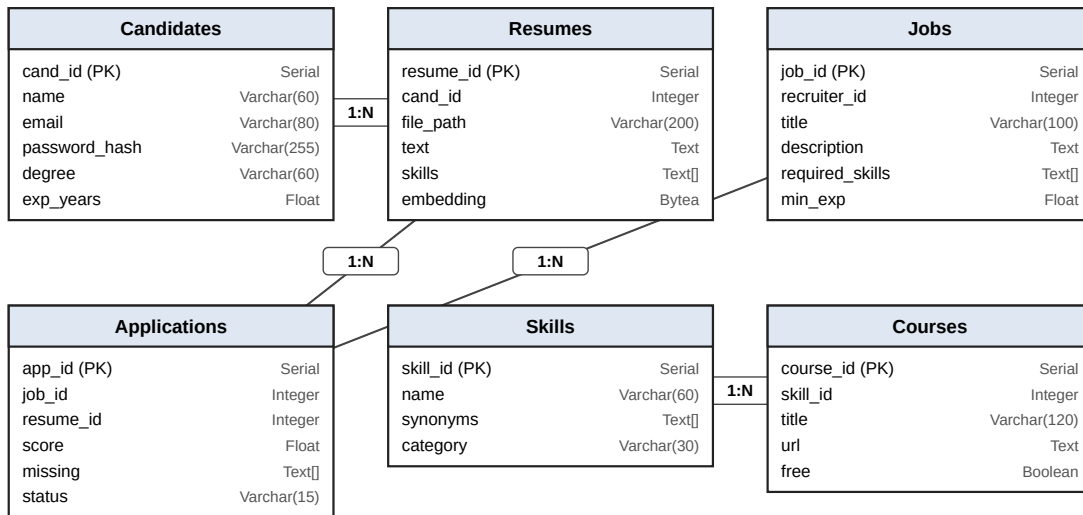


Fig 5: Entity relationship diagram

6.5 Table Design

CANDIDATES

FIELD NAME	DATA TYPE	DESCRIPTION
cand_id	Serial	Primary key
name	Varchar(60)	Name
email	Varchar(80)	E-mail (login)
password_hash	Varchar(255)	Hashed password
degree	Varchar(60)	Highest degree
exp_years	Float	Experience

RESUMES

FIELD NAME	DATA TYPE	DESCRIPTION
resume_id	Serial	Primary key
cand_id	Integer	Foreign key to Candidates
file_path	Varchar(200)	Stored file
text	Text	Extracted text
skills	Text[]	Normalised skills
embedding	Bytea	SBERT vector

JOBS

FIELD NAME	DATA TYPE	DESCRIPTION
job_id	Serial	Primary key
recruiter_id	Integer	Recruiter
title	Varchar(100)	Job title
description	Text	Job description
required_skills	Text[]	Required skills
min_exp	Float	Minimum experience

APPLICATIONS

FIELD NAME	DATA TYPE	DESCRIPTION
app_id	Serial	Primary key
job_id	Integer	Foreign key to Jobs
resume_id	Integer	Foreign key to Resumes
score	Float	Final score
missing	Text[]	Missing skills
status	Varchar(15)	applied / shortlisted / rejected

SKILLS

FIELD NAME	DATA TYPE	DESCRIPTION
skill_id	Serial	Primary key
name	Varchar(60)	Canonical skill
synonyms	Text[]	Alternative names
category	Varchar(30)	Category

COURSES

FIELD NAME	DATA TYPE	DESCRIPTION
course_id	Serial	Primary key
skill_id	Integer	Foreign key to Skills
title	Varchar(120)	Course title

url	Text	Link
free	Boolean	Free course

6.6 Algorithm Design

The scoring algorithm combines semantic similarity with explicit skill and eligibility matching to give an explainable score.

1. **Parse resume (PDF/DOCX) into sections** – pdfplumber / python-docx extract text; headings such as 'Education', 'Skills', 'Projects' split the sections.
2. **Extract skills, education and experience using NER** – spaCy EntityRuler patterns built from the ontology plus regex for years of experience and CGPA.
3. **Normalise skills with a skill ontology** – Synonyms are mapped to canonical names (e.g. 'sklearn' -> 'scikit-learn').
4. **Embed resume and job description with Sentence-BERT** – The all-MiniLM-L6-v2 model produces 384-d vectors for each section and the JD.
5. **Compute cosine similarity score** – Semantic score = cosine similarity between resume and JD embeddings.
6. **Identify skill gaps** – Missing = required skills - candidate skills.
7. **Recommend courses and jobs** – Courses linked to missing skills; jobs ranked by score.
8. **Rank candidates for recruiter view** – Final score = 0.5 x semantic + 0.35 x skill coverage + 0.15 x eligibility.

PSEUDO CODE

```
SCORE(resume, job):
  text <- EXTRACT_TEXT(resume); sections <- SPLIT_BY_HEADINGS(text)
  skills <- NORMALISE(NER_SKILLS(text))
  sem <- COSINE(SBERT(sections.skills + sections.projects +
sections.experience), SBERT(job.description))
  cov <- |skills ∩ job.required| / |job.required|
  elig <- 1 if exp >= job.min_exp and degree matches else 0.5
  final <- 100 * (0.5*sem + 0.35*cov + 0.15*elig)
  missing <- job.required - skills
  RETURN final, skills ∩ job.required, missing, COURSES(missing)
```

6.7 Modules

The modules are grouped by the Candidate, Recruiter and Admin users.

MODULES

CANDIDATE

1. REGISTER AND UPLOAD RESUME
2. VIEW ATS SCORE
3. VIEW SKILL GAPS AND COURSES
4. GET JOB RECOMMENDATIONS

RECRUITER

1. POST JOB
2. VIEW RANKED CANDIDATES

ADMIN

1. MANAGE SKILL ONTOLOGY
2. MANAGE COURSES

CANDIDATE

REGISTER AND UPLOAD RESUME

The candidate creates an account and uploads a resume in PDF or DOCX format. The parsed details are shown for confirmation.

VIEW ATS SCORE

The candidate selects a target job role or pastes a job description and receives an ATS score with a breakdown of semantic match, skill match and experience.

VIEW SKILL GAPS AND COURSES

Missing skills are listed with recommended free courses (NPTEL, Coursera, YouTube playlists).

GET JOB RECOMMENDATIONS

Jobs with the highest match scores are recommended, and the candidate can apply directly.

RECRUITER

POST JOB

The recruiter posts a job description with required and preferred skills, minimum experience and qualification.

VIEW RANKED CANDIDATES

All applicants are ranked by score with matched and missing skills highlighted; the recruiter can shortlist or reject.

ADMIN

MANAGE SKILL ONTOLOGY

Admin adds skills and synonyms, for example 'ML' = 'Machine Learning'.

MANAGE COURSES

Admin maintains the course catalogue linked to skills.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

HireMatch was implemented with a FastAPI back end in Python and a React front end. Resume text extraction uses pdfplumber for PDF and python-docx for Word files. The NLP pipeline uses spaCy en_core_web_sm with an EntityRuler containing about 1,800 skill patterns derived from ESCO and common job postings.

7.2 Implementation

Scoring weights were tuned using 300 resume-job pairs labelled by placement officers as Good, Average or Poor fit. Embeddings are cached in the database so that re-ranking all applicants for a job takes only matrix multiplication.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.
- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	Upload PDF resume	resume.pdf	Sections and skills extracted	Pass
TC02	Upload unsupported file	resume.jpg	Error 'Upload PDF or DOCX'	Pass
TC03	Synonym handling	Resume says 'ML, sklearn'	Mapped to Machine Learning, scikit-learn	Pass
TC04	ATS score	Data analyst JD	Score with breakdown	Pass
TC05	Skill gap	JD needs Power BI	Power BI listed missing with course	Pass
TC06	Recruiter ranking	120 applicants	Ranked list in < 10 s	Pass
TC07	Bias protection	Same resume, different name	Same score	Pass
TC08	Job recommendation	Python + SQL resume	Data/backend jobs recommended	Pass

PERFORMANCE EVALUATION

Agreement of system ranking with placement officer labels (300 pairs):

Method	Accuracy (Good/Avg/Poor)	Spearman correlation
Keyword count (ATS baseline)	61%	0.48
TF-IDF cosine	68%	0.57
SBERT only	76%	0.69
SBERT + skill coverage (proposed)	84%	0.78

7.4 Source Code

parser.py

```
import re, pdfplumber, docx, spacy, json

nlp = spacy.load("en_core_web_sm")
```

```

ontology = json.load(open("data/skills.json")) # {"Machine Learning":
["ml", "machine learning"], ...}
ruler = nlp.add_pipe("entity_ruler", before="ner")
ruler.add_patterns([{"label": "SKILL", "pattern": [{"LOWER": t} for t in
syn.split()], "id": canon}
                    for canon, syns in ontology.items() for syn in syns])
HEADINGS = ["education", "skills", "technical skills", "experience",
"internship",
            "projects", "certifications", "achievements"]

def extract_text(path):
    if path.endswith(".pdf"):
        with pdfplumber.open(path) as pdf:
            return "\n".join(p.extract_text() or "" for p in pdf.pages)
    if path.endswith(".docx"):
        return "\n".join(p.text for p in docx.Document(path).paragraphs)
    raise ValueError("Upload PDF or DOCX")

def split_sections(text):
    sections, cur = {"header": []}, "header"
    for line in text.splitlines():
        key = line.strip().lower().rstrip(":")
        if key in HEADINGS:
            cur = key; sections.setdefault(cur, [])
        else:
            sections[cur].append(line)
    return {k: "\n".join(v) for k, v in sections.items()}

def parse(path):
    text = extract_text(path)
    doc = nlp(text)
    skills = sorted({e.ent_id_ for e in doc.ents if e.label_ == "SKILL"})
    years = re.findall(r"(\d+(?:\.\d)?)\+?\s*(?:years|yrs)", text.lower())
    cgpa = re.search(r"(:cgpa|gpa)[^\d]*(\d\.\d+)", text.lower())
    return {"text": text, "sections": split_sections(text), "skills": skills,
            "experience": max(map(float, years)) if years else 0.0,
            "cgpa": float(cgpa.group(1)) if cgpa else None}

```

scorer.py

```

from sentence_transformers import SentenceTransformer, util
from parser import nlp

model = SentenceTransformer("all-MiniLM-L6-v2")

def jd_skills(jd_text):
    return sorted({e.ent_id_ for e in nlp(jd_text).ents if e.label_ == "SKILL"})

def score(resume, job):
    focus = " ".join(resume["sections"].get(k, "") for k in ("skills",
"technical skills", "projects", "experience", "internship"))
    sem = float(util.cos_sim(model.encode(focus or resume["text"]),
model.encode(job["description"])))
    required = set(job.get("required_skills") or jd_skills(job["description"]))
    have = set(resume["skills"])
    coverage = len(have & required) / len(required) if required else 1.0
    elig = 1.0 if resume["experience"] >= job.get("min_exp", 0) else 0.5
    final = round(100 * (0.5 * max(sem, 0) + 0.35 * coverage + 0.15 * elig), 1)
    return {"score": final, "semantic": round(sem, 3), "coverage":
round(coverage, 2),
            "matched": sorted(have & required), "missing": sorted(required -

```

```

have)}}

def rank(resumes, job):
    results = [(rid, score(r, job)) for rid, r in resumes.items()]
    return sorted(results, key=lambda x: x[1]["score"], reverse=True)

```

7.5 Screen Layout

SCREENSHOTS:

RESUME UPLOAD

https://hirematch.app/

HireMatch [My Resume](#) [ATS Score](#) [Jobs](#) [Recruiter](#) [Courses](#)

Upload your resume

Resume (PDF / DOCX)

Karthik_S_Resume.pdf (186 KB)

Target role

Data Analyst - Fresher

Or paste a job description

Looking for a Data Analyst with Python, SQL, Excel, Power BI ...

Analyse Resume

Extracted details

Degree: B.Sc. Computer Science (CGPA 8.4)

Skills: Python, SQL, Excel, Pandas, Machine Learning, Tableau

Projects: 3 Internship: 1 (2 months)

ATS SCORE AND SKILL GAPS

https://hirematch.app/

HireMatch

[My Resume](#)[ATS Score](#)[Jobs](#)[Recruiter](#)[Courses](#)

ATS Score - Data Analyst (Fresher)

ATS Score 72 / 100 - Good, can improve

Semantic match	0.71
Required skills matched	5 of 7 (Python, SQL, Excel, Pandas, Tableau)
Missing skills	Power BI, Statistics
Eligibility	Fresher role - eligible

Add a project using Power BI dashboards - Free course: Microsoft Learn PL-300 path

Mention statistics coursework (hypothesis testing, regression) - NPTEL 'Statistics for Data Science'

Use action verbs and numbers: 'Reduced report time by 40%'.

JOB RECOMMENDATIONS

https://hirematch.app/

HireMatch

[My Resume](#)[ATS Score](#)[Jobs](#)[Recruiter](#)[Courses](#)

Jobs recommended for you

Location: Chennai / Remote

Experience: 0-1 yr

Job	Company	Location	Match	Action
Junior Data Analyst	Insight Analytics	Chennai	84%	Apply
Business Analyst Trainee	RetailSoft	Coimbatore	79%	Apply
Python Developer (Fresher)	CodeNest	Remote	68%	Apply
MIS Executive	FinServe	Madurai	63%	Apply

RECRUITER - RANKED CANDIDATES

https://hirematch.app/

HireMatch

[My Resume](#)[ATS Score](#)[Jobs](#)[Recruiter](#)[Courses](#)

Junior Data Analyst - 146 applicants ranked

Min score: 60

Status: All

Rank	Candidate	Score	Matched	Missing
1	Candidate #A21	88	Python, SQL, Power BI, Excel	-
2	Candidate #B07	84	Python, SQL, Tableau, Stats	Power BI
3	Candidate #A55	81	SQL, Excel, Power BI	Python
4	Candidate #C12	72	Python, SQL, Excel	Power BI, Stats

CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The AI Resume Screening and Job Recommendation Platform understands resumes and job descriptions semantically, achieving 84% agreement with placement officers compared with 61% for keyword-based ATS screening. Candidates receive transparent scores, skill gaps and course suggestions, while recruiters receive ranked and explained candidate lists.

By excluding personal attributes from scoring, the system also promotes fairer screening. The platform can be used by college placement cells to improve fresher employability and by small companies to speed up hiring.

8.2 Future Enhancement

- Resume rewriting suggestions using large language models.
- Interview question generation based on the candidate's projects.
- Tamil and other Indian language resume support.
- Integration with LinkedIn and job portals via APIs.
- Bias auditing dashboard for recruiters.

CHAPTER IX

BIBLIOGRAPHY

1. Robertson, S., and Zaragoza, H., "The Probabilistic Relevance Framework: BM25 and Beyond", Foundations and Trends in Information Retrieval, Vol. 3, No. 4, 2009.
2. Mikolov, T., Chen, K., Corrado, G., and Dean, J., "Efficient Estimation of Word Representations in Vector Space", arXiv:1301.3781, 2013.
3. Reimers, N., and Gurevych, I., "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", Proc. EMNLP-IJCNLP, 2019.
4. Honnibal, M., Montani, I., Van Landeghem, S., and Boyd, A., spaCy: Industrial-strength Natural Language Processing in Python, 2020.
5. Raghavan, M., Barocas, S., Kleinberg, J., and Levy, K., "Mitigating Bias in Algorithmic Hiring: Evaluating Claims and Practices", Proc. ACM FAT*, 2020.
6. Jurafsky, D., and Martin, J. H., Speech and Language Processing, 3rd Edition draft, Stanford University.

WEBSITE REFERENCE

1. <https://spacy.io>
2. <https://www.sbert.net>
3. <https://fastapi.tiangolo.com>
4. <https://esco.ec.europa.eu>